



GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING SCIENCES AND
TECHNOLOGY

IC Design Verification

Multi-Cycle RISC-V Processor Implementation

Complete FSM-Based Control Unit, Datapath Analysis and Simulation
Verification

Submitted To: Eng Abbas Khan

Submitted By: Abdul Samad Abbasi
Student ID: 25ins-01030

Course: Computer Architecture – Module 2

Date: June 24, 2026

Abstract

This report presents a comprehensive design, implementation, verification, and analysis of a Multi-Cycle RISC-V Processor implemented in Verilog HDL. The processor executes a representative subset of the RV32I base integer instruction set: R-type arithmetic (ADDI, SUB), I-type load (LW), S-type store (SW), B-type branch (BEQ), and J-type unconditional jump (JAL). A distinguishing feature of the multi-cycle architecture is its Finite State Machine (FSM)-based control unit, which sequences each instruction through only the pipeline stages it requires, thereby reducing the average clock-cycle penalty while sharing functional units across different instruction types. This report details every aspect of the design: the top-level datapath, individual module implementations complete with Verilog source listings, per-instruction FSM traversal, simulation waveform analysis with annotated signal discussion, a rigorous comparison between single-cycle and multi-cycle designs, and tabulated control-signal truth tables. All results are validated against Xilinx Vivado 2018.2 behavioural simulation.

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Design Philosophy	3
1.3	Key Features	3
2	Architecture and Datapath	3
2.1	Top-Level Datapath	3
2.2	Module Hierarchy	4
2.3	Pipeline Registers	5
3	Control Unit: Finite State Machine	5
3.1	FSM Overview	5
3.2	Complete FSM Diagram	6
3.3	Control Signal Truth Table	7
4	Instruction-Level FSM Traversal	7
4.1	R-Type Instructions (SUB)	7
4.1.1	State Path	7
4.1.2	Stage-by-Stage Description	7
4.1.3	Control Signal Summary	8
4.2	I-Type ALU Instructions (ADDI)	8
4.2.1	State Path	8
4.2.2	Stage-by-Stage Description	8
4.2.3	Control Signal Summary	9
4.3	LW – Load Word	9
4.3.1	State Path	9
4.3.2	Stage-by-Stage Description	9
4.3.3	Control Signal Summary	9
4.4	SW – Store Word	9
4.4.1	State Path	9
4.4.2	Stage-by-Stage Description	10
4.4.3	Control Signal Summary	10
4.5	BEQ – Branch Equal	10
4.5.1	State Path	10
4.5.2	Stage-by-Stage Description	10
4.5.3	Control Signal Summary	11
4.6	JAL – Jump and Link	11
4.6.1	State Path	11
4.6.2	Stage-by-Stage Description	11
4.6.3	Control Signal Summary	11

5	Simulation Results	12
5.1	Test Program	12
5.2	TCL Console State Trace	12
5.3	Complete Simulation Waveform	13
5.3.1	ADDI / I-Type – PC = 0	13
5.3.2	SUB / R-Type – PC = 4	13
5.3.3	LW – PC = 8	14
5.3.4	SW – PC = 12	14
5.3.5	BEQ – PC = 16	14
5.3.6	JAL – PC = 20, Jump to 8	14
6	Multi-Cycle vs. Single-Cycle: Comparative Analysis	15
6.1	Architectural Differences	15
6.2	Advantages of the Multi-Cycle Design	15
6.3	Disadvantages of the Multi-Cycle Design	16
6.4	Performance Comparison	16
7	Summary and Conclusion	17
7.1	Verification Summary	17
7.2	Key Observations	17
7.3	Conclusion	17
	Appendix A Complete Verilog Source Code	18
A.1	Top-Level Module	18
A.2	FSM Control Unit	19
A.3	ALU Decoder	21
A.4	Immediate Generator	21
A.5	Testbench	22

List of Figures

1	Multi-Cycle RISC-V Datapath with JAL state signal values annotated (source: Harris & Harris).	4
2	Complete 11-State Moore FSM for the Multi-Cycle RISC-V Control Unit.	6
3	Tcl Console output: PC, Instruction, and FSM State on every clock edge.	12
4	Complete Vivado Simulation Waveform (0–550 ns), showing all datapath and control signals.	13

List of Tables

1	Module Hierarchy and Functions	5
2	Inter-Stage Pipeline Registers	5

3	FSM State Definitions	6
4	Control Signal Outputs per FSM State	7
5	R-Type (SUB) Control Signals per Stage	8
6	I-Type (ADDI) Control Signals per Stage	9
7	LW Control Signals per Stage	9
8	SW Control Signals per Stage	10
9	BEQ Control Signals per Stage	11
10	JAL Control Signals per Stage	11
11	Test Program Loaded in Instruction Memory	12
12	Structural Comparison: Single-Cycle vs. Multi-Cycle	15
13	Instruction Verification Summary	17

1 Introduction

1.1 Project Overview

The RISC-V Instruction Set Architecture (ISA) is a free and open-source ISA that has gained rapid adoption in academia, industry, and research communities. Unlike proprietary architectures, RISC-V allows unrestricted implementation, making it an ideal platform for studying processor micro-architecture. This project implements a Multi-Cycle RISC-V processor supporting the following instruction types drawn from the RV32I base integer specification:

- **R-Type:** Register-to-register arithmetic (SUB)
- **I-Type:** Immediate arithmetic (ADDI) and Load Word (LW)
- **S-Type:** Store Word (SW)
- **B-Type:** Branch Equal (BEQ)
- **J-Type:** Jump and Link (JAL)

1.2 Design Philosophy

The multi-cycle architecture partitions each instruction's execution into a sequence of discrete stages, each occupying one clock cycle. A Moore-type FSM governs the processor, asserting only the control signals needed for the current stage. Functional units (ALU, memory, register file) are shared between stages rather than replicated. As a result, simple instructions complete in fewer cycles, hardware cost is reduced through resource sharing, and the critical path per cycle is shorter, enabling a potentially higher clock frequency than a single-cycle design.

1.3 Key Features

- 32-bit RV32I architecture with 32 general-purpose registers
- Unified instruction and data memory (separated by address range)
- 11-state Moore FSM control unit
- Intermediate pipeline registers: IR, OldPC, A, WriteData, ALUOut, Data
- 3:1 multiplexers on ALU inputs to support PC+4, branch-target, and data-path computation using a single shared ALU
- Complete Verilog RTL targeting Xilinx Vivado 2018.2

2 Architecture and Datapath

2.1 Top-Level Datapath

Figure 1 shows the multi-cycle datapath as drawn in Harris & Harris [1], reproduced here for reference. The key structural difference from a single-cycle design is the

presence of inter-stage pipeline registers that latch intermediate values between clock edges, allowing a single ALU and a single memory to be reused across multiple cycles.

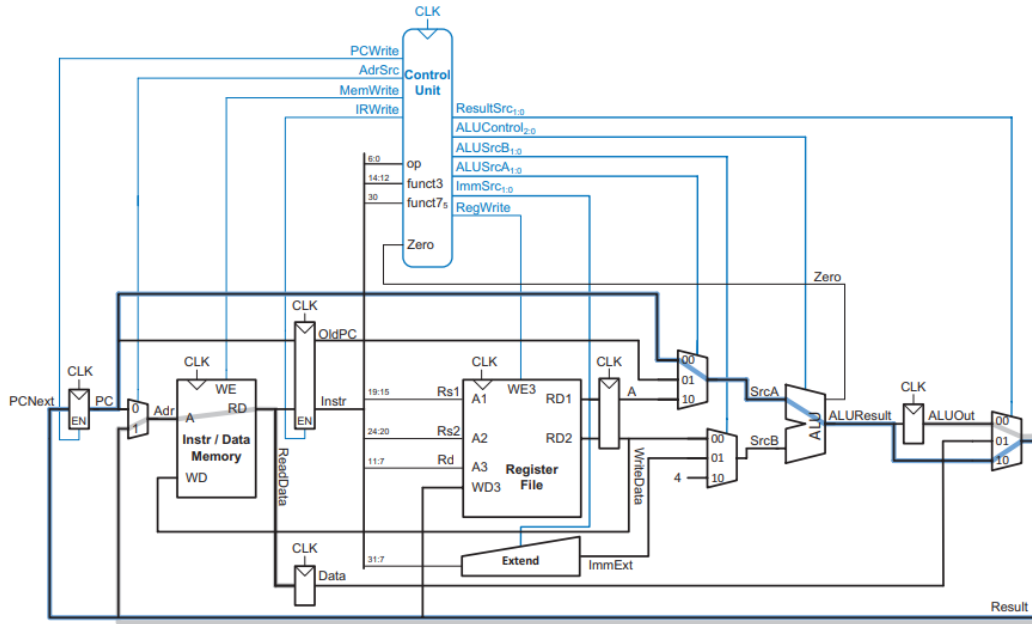


Figure 1: Multi-Cycle RISC-V Datapath with JAL state signal values annotated (source: Harris & Harris).

In the Fetch state the PC is used as the memory address and the instruction is latched into the Instruction Register (IR) and OldPC. In subsequent states, the decoder reads from IR while the datapath uses OldPC for branch and jump target calculations. The unified memory is time-multiplexed: it serves instruction fetch in the Fetch state and data read/write in the MemRead/MemWrite states, selected by the AdrSrc mux.

2.2 Module Hierarchy

Table 1 lists every Verilog module in the design together with its type and function.

Table 1: Module Hierarchy and Functions

Module	Function	Type
<code>top_module</code>	Top-level integration	Structural
<code>pc_32bit</code>	Program counter with enable	Sequential
<code>mux_2x1</code>	2:1 address mux (AdrSrc)	Combinational
<code>Inst_DM</code>	Unified instruction/data memory (1 KB)	Sequential
<code>Instr</code>	IR + OldPC pipeline register	Sequential
<code>Data</code>	Data pipeline register	Sequential
<code>REG_FILE</code>	32×32-bit register file	Sequential
<code>Imm_Gen</code>	Immediate sign-extension	Combinational
<code>RF_Reg</code>	A / WriteData pipeline registers	Sequential
<code>mux_3x1</code>	3:1 mux (SrcA, SrcB, Result)	Combinational
<code>ALU</code>	32-bit arithmetic logic unit	Combinational
<code>ALU_Reg</code>	ALUOut pipeline register	Sequential
<code>control_unit</code>	Top-level control wrapper	Mixed
<code>fsm</code>	11-state Moore FSM	Sequential
<code>alu_decoder</code>	ALUControl generation	Combinational
<code>extender_decoder</code>	ImmSrc selection	Combinational

2.3 Pipeline Registers

Unlike a single-cycle design, the multi-cycle processor inserts registers between stages so that values computed in one cycle are available combinational in the next without re-reading memory or the register file. Table 2 lists these registers.

Table 2: Inter-Stage Pipeline Registers

Register	Holds	Update Condition
PC	Current instruction address	PCWrite = 1
OldPC / IR	Fetch instruction and prior PC	IRWrite = 1
A, WriteData	RS1 and RS2 values from register file	Every cycle
ALUOut	Previous ALU result	Every cycle
Data	Memory read data	Every cycle

3 Control Unit: Finite State Machine

3.1 FSM Overview

The control unit is a Moore Finite State Machine with 11 states. Outputs depend only on the current state, with the exception of the next-state logic, which inspects the instruction opcode decoded from IR[6:0]. The PCWrite signal is derived combinational from the FSM outputs: `PCWrite = PCUpdate OR (Branch AND Zero)`, where Zero is the ALU zero flag.

Table 3 defines each state.

Table 3: FSM State Definitions

State #	Name	Purpose
0	Fetch	Fetch instruction from memory; increment PC to PC+4
1	Decode	Decode opcode; pre-compute potential branch or jump target
2	MemAdr	Compute load/store effective address via ALU
3	MemRead	Issue read to data memory using ALUOut as address
4	MemWB	Write loaded data from Data register to register file
5	MemWrite	Issue write to data memory (SW)
6	ExecuteR	Perform R-type ALU operation
7	ALUWB	Write ALU result (ALUOut) to register file
8	ExecuteI	Perform I-type immediate ALU operation
9	JAL	Update PC to jump target; compute return address
10	BEQ	Compare rs1 and rs2; conditionally update PC

3.2 Complete FSM Diagram

Figure 2 shows the complete state transition diagram. Arcs are labelled with the opcode condition that causes a particular transition out of the Decode state; all other transitions are unconditional.

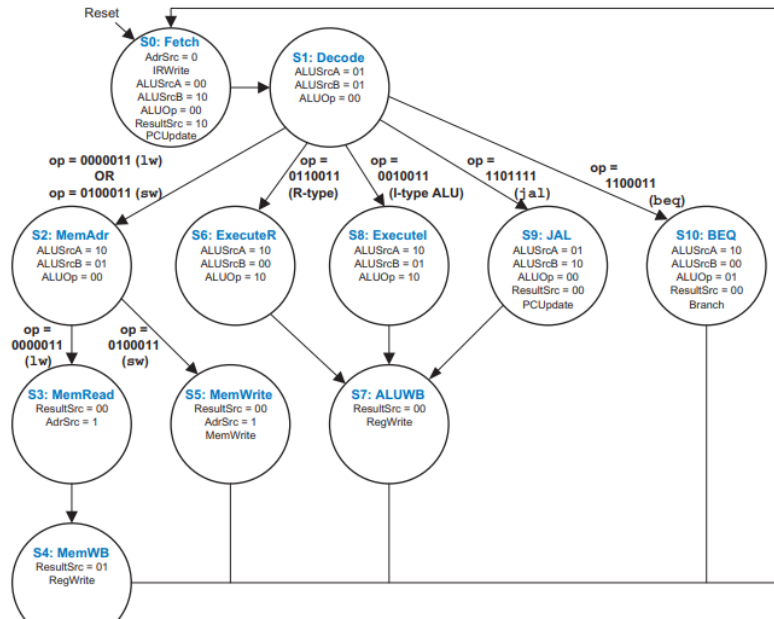


Figure 2: Complete 11-State Moore FSM for the Multi-Cycle RISC-V Control Unit.

3.3 Control Signal Truth Table

Table 4 gives the output values for every control signal in every FSM state. Dashes indicate don't-care (the signal defaults to 0).

Table 4: Control Signal Outputs per FSM State

State	PCUpd	Branch	AdrSrc	IRWr	RegWr	MemWr	ALUSrcA	ALUSrcB	ALUOp	ResSrc
Fetch	1	0	0	1	0	0	00	10	00	10
Decode	0	0	0	0	0	0	01	01	00	–
MemAdr	0	0	0	0	0	0	10	01	00	–
MemRead	0	0	1	0	0	0	–	–	–	00
MemWB	0	0	0	0	1	0	–	–	–	01
MemWrite	0	0	1	0	0	1	–	–	–	00
ExecuteR	0	0	0	0	0	0	10	00	10	–
ALUWB	0	0	0	0	1	0	–	–	–	00
ExecuteI	0	0	0	0	0	0	10	01	10	–
JAL	1	0	0	0	0	0	01	10	00	00
BEQ	0	1	0	0	0	0	10	00	01	00

ALUSrcA encoding: 00 = PC, 01 = OldPC, 10 = A (rs1 latch).

ALUSrcB encoding: 00 = WriteData (rs2 latch), 01 = ImmExt, 10 = 32'd4.

ResultSrc encoding: 00 = ALUOut, 01 = Data register, 10 = ALUResult (combinational).

4 Instruction-Level FSM Traversal

Each instruction type follows a unique path through the FSM. The following subsections describe each path, the states visited, the total cycle count, and the key operations performed at every stage.

4.1 R-Type Instructions (SUB)

4.1.1 State Path

$$S_0 \text{ (Fetch)} \rightarrow S_1 \text{ (Decode)} \rightarrow S_6 \text{ (ExecuteR)} \rightarrow S_7 \text{ (ALUWB)} \rightarrow S_0$$

Total cycles required: 4.

4.1.2 Stage-by-Stage Description

S0 – Fetch: IRWrite and PCUpdate are both asserted. The PC is presented to the unified memory and the returned instruction word is latched into the IR; simultaneously the ALU computes PC+4 (ALUSrcA=00 selects PC, ALUSrcB=10 selects the constant 4, ALUOp=00 forces ADD) and ResultSrc=10 routes the combinational ALU result back to the PC via the result mux, so the PC register advances.

S1 – Decode: The opcode is decoded as 0110011 (R-type). RS1 and RS2 values are read from the register file and captured in the A and WriteData pipeline registers. The

ALU pre-computes OldPC + ImmExt (branch target), but this value is not used for R-type instructions.

S6 – ExecuteR: ALUSrcA=10 routes the A register (rs1) to the ALU; ALUSrcB=00 routes the WriteData register (rs2). ALUOp=10 instructs the ALU decoder to use funct3 and funct7[5] to select the operation. For SUB: funct3=000, funct7[5]=1, op[5]=1, so ALUControl=001 (subtract). The result is stored in the ALUOut register.

S7 – ALUWB: RegWrite is asserted and ResultSrc=00 routes ALUOut to the register-file write port. The subtraction result is written to rd.

4.1.3 Control Signal Summary

Table 5: R-Type (SUB) Control Signals per Stage

State	PCUpd	IRWr	RegWr	MemWr	ALUSrcA	ALUSrcB	ALUOp	ResSrc	ALUCtrl
Fetch	1	1	0	0	00	10	00	10	000
Decode	0	0	0	0	01	01	00	–	000
ExecuteR	0	0	0	0	10	00	10	–	001
ALUWB	0	0	1	0	–	–	–	00	–

4.2 I-Type ALU Instructions (ADDI)

4.2.1 State Path

$$S_0 \rightarrow S_1 \rightarrow S_8 \text{ (ExecuteI)} \rightarrow S_7 \text{ (ALUWB)} \rightarrow S_0$$

Total cycles required: 4.

4.2.2 Stage-by-Stage Description

States S0 and S1 are identical to R-type. In S1 the opcode is decoded as 0010011 (I-type), so the FSM transitions to S8 instead of S6.

S8 – ExecuteI: ALUSrcA=10 (A register / rs1), ALUSrcB=01 (ImmExt). ALUOp=10 causes the ALU decoder to select the operation from funct3, exactly as for R-type, but with the immediate as the second operand. For ADDI: funct3=000, funct7[5]=0, so ALUControl=000 (ADD). Result stored in ALUOut.

S7 – ALUWB: RegWrite=1, ResultSrc=00. ALUOut (rs1 + imm) written to rd. Behaviour is identical to R-type write-back.

4.2.3 Control Signal Summary

Table 6: I-Type (ADDI) Control Signals per Stage

State	PCUpd	IRWr	RegWr	MemWr	ALUSrcA	ALUSrcB	ALUOp	ResSrc	ALUCtrl
Fetch	1	1	0	0	00	10	00	10	000
Decode	0	0	0	0	01	01	00	–	000
ExecuteI	0	0	0	0	10	01	10	–	000
ALUWB	0	0	1	0	–	–	–	00	–

4.3 LW – Load Word

4.3.1 State Path

$$S_0 \rightarrow S_1 \rightarrow S_2 \text{ (MemAdr)} \rightarrow S_3 \text{ (MemRead)} \rightarrow S_4 \text{ (MemWB)} \rightarrow S_0$$

Total cycles required: **5**.

4.3.2 Stage-by-Stage Description

S2 – MemAdr: ALUSrcA=10 (A / rs1), ALUSrcB=01 (ImmExt with ImmSrc=00 for I-type format), ALUOp=00 (ADD). Effective address = $rs1 + \text{sign_ext}(\text{imm12})$ is computed and stored in ALUOut.

S3 – MemRead: AdrSrc=1 routes ALUOut to the unified memory address bus, bypassing the PC. ResultSrc=00 ensures ALUOut is available on the result bus. The memory read is issued; the returned word is captured in the Data pipeline register at the next rising edge.

S4 – MemWB: RegWrite=1, ResultSrc=01 selects the Data register as the write-back source. The loaded word is written to the destination register rd.

4.3.3 Control Signal Summary

Table 7: LW Control Signals per Stage

State	PCUpd	IRWr	RegWr	MemWr	AdrSrc	ALUSrcA	ALUSrcB	ALUOp	ResSrc
Fetch	1	1	0	0	0	00	10	00	10
Decode	0	0	0	0	0	01	01	00	–
MemAdr	0	0	0	0	0	10	01	00	–
MemRead	0	0	0	0	1	–	–	–	00
MemWB	0	0	1	0	0	–	–	–	01

4.4 SW – Store Word

4.4.1 State Path

$$S_0 \rightarrow S_1 \rightarrow S_2 \text{ (MemAdr)} \rightarrow S_5 \text{ (MemWrite)} \rightarrow S_0$$

Total cycles required: **4**.

4.4.2 Stage-by-Stage Description

States S0, S1, and S2 proceed identically to LW. In S2 the immediate is the S-type encoding (ImmSrc=01), so the extender concatenates bits [31:25] and [11:7] to form the 12-bit signed offset.

S5 – MemWrite: MemWrite=1 enables a synchronous write to memory. AdrSrc=1 routes ALUOut (the computed address) to the memory. The write data comes from the WriteData register, which holds the value of rs2 captured in the Decode state.

4.4.3 Control Signal Summary

Table 8: SW Control Signals per Stage

State	PCUpd	IRWr	RegWr	MemWr	AdrSrc	ALUSrcA	ALUSrcB	ALUOp	ResSrc
Fetch	1	1	0	0	0	00	10	00	10
Decode	0	0	0	0	0	01	01	00	–
MemAdr	0	0	0	0	0	10	01	00	–
MemWrite	0	0	0	1	1	–	–	–	00

4.5 BEQ – Branch Equal

4.5.1 State Path

$$S_0 \rightarrow S_1 \rightarrow S_{10} \text{ (BEQ)} \rightarrow S_0$$

Total cycles required: **3**.

4.5.2 Stage-by-Stage Description

S1 – Decode (critical for BEQ): ImmSrc=10 selects the B-type immediate format. The ALU computes OldPC + sign_ext(B-imm), which is the branch target address. This result is stored in ALUOut and later routed to the PC if the branch is taken.

S10 – BEQ: ALUSrcA=10 (A / rs1), ALUSrcB=00 (WriteData / rs2), ALUOp=01 (forces ALUControl=001, subtraction). The ALU computes rs1 – rs2. If the result is zero the Zero flag asserts, causing PCWrite = Branch AND Zero = 1 AND 1 = 1, and ResultSrc=00 routes the previously computed branch target (from ALUOut, stored from the Decode stage) to the PC. If Zero=0 the branch is not taken and PC remains at PC+4.

4.5.3 Control Signal Summary

Table 9: BEQ Control Signals per Stage

State	PCUpd	Branch	IRWr	RegWr	MemWr	ALUSrcA	ALUSrcB	ALUOp	ResSrc	ALUCtrl
Fetch	1	0	1	0	0	00	10	00	10	000
Decode	0	0	0	0	0	01	01	00	–	000
BEQ	0	1	0	0	0	10	00	01	00	001

4.6 JAL – Jump and Link

4.6.1 State Path

$$S_0 \rightarrow S_1 \rightarrow S_9 \text{ (JAL)} \rightarrow S_7 \text{ (ALUWB)} \rightarrow S_0$$

Total cycles required: 4.

4.6.2 Stage-by-Stage Description

S1 – Decode (critical for JAL): ImmSrc=11 selects the J-type immediate format. The ALU computes $\text{OldPC} + \text{sign_ext}(\text{J-imm})$, the jump target address, and stores it in ALUOut.

S9 – JAL: PCUpdate=1 is asserted. ResultSrc=00 routes ALUOut (the jump target) through the result mux back to the PC input; on the next rising edge the PC updates to the target. Meanwhile ALUSrcA=01 (OldPC) and ALUSrcB=10 (constant 4) with ALUOp=00 cause the ALU to compute $\text{OldPC}+4$ (the return address); this result is stored in the ALUOut register (overwriting the jump target that has already been captured by the PC).

S7 – ALUWB: RegWrite=1, ResultSrc=00. The return address ($\text{OldPC}+4$) now in ALUOut is written to the link register rd.

4.6.3 Control Signal Summary

Table 10: JAL Control Signals per Stage

State	PCUpd	IRWr	RegWr	MemWr	ALUSrcA	ALUSrcB	ALUOp	ResSrc	ALUCtrl
Fetch	1	1	0	0	00	10	00	10	000
Decode	0	0	0	0	01	01	00	–	000
JAL	1	0	0	0	01	10	00	00	000
ALUWB	0	0	1	0	–	–	–	00	–

5 Simulation Results

5.1 Test Program

The processor was verified with the six-instruction sequence in Table 11, loaded into unified memory starting at byte address 0x00. Data memory begins at address 128 (0x80), and a pre-loaded word 0x12345678 sits at address 140 (the target of the LW instruction when x20 = 128).

Table 11: Test Program Loaded in Instruction Memory

PC	Encoding (hex)	Assembly	Type	Cycles
0	00a85513	addi x10, x16, 10	I	4
4	40f706b3	sub x13, x14, x15	R	4
8	00ca2983	lw x19, 12(x20)	I-load	5
12	013a2823	sw x19, 16(x20)	S	4
16	00d50663	beq x10, x13, +12	B	3
20	ff5ff16f	jal x2, -12	J	4

5.2 TCL Console State Trace

The testbench prints the PC, instruction register, and current FSM state number on every rising clock edge after reset. Figure 3 shows the captured Tcl console output from Vivado.

```

Tcl Console x Messages Log
[Icons]
PC=0 INST=0x00000000 STATE=0
PC=4 INST=0x00a85513 STATE=1
PC=4 INST=0x00a85513 STATE=8
PC=4 INST=0x00a85513 STATE=7
PC=4 INST=0x00a85513 STATE=0
PC=8 INST=0x40f706b3 STATE=1
PC=8 INST=0x40f706b3 STATE=6
PC=8 INST=0x40f706b3 STATE=7
PC=8 INST=0x40f706b3 STATE=0
PC=12 INST=0x00ca2983 STATE=1
PC=12 INST=0x00ca2983 STATE=2
PC=12 INST=0x00ca2983 STATE=3
PC=12 INST=0x00ca2983 STATE=4
PC=12 INST=0x00ca2983 STATE=0
PC=16 INST=0x013a2823 STATE=1
PC=16 INST=0x013a2823 STATE=2
PC=16 INST=0x013a2823 STATE=5
PC=16 INST=0x013a2823 STATE=0
PC=20 INST=0x00d50663 STATE=1
PC=20 INST=0x00d50663 STATE=10
PC=20 INST=0x00d50663 STATE=0
PC=24 INST=0xff5ff16f STATE=1
PC=24 INST=0xff5ff16f STATE=9
PC=8 INST=0xff5ff16f STATE=7
PC=8 INST=0xff5ff16f STATE=0
PC=12 INST=0x00ca2983 STATE=1

```

Figure 3: Tcl Console output: PC, Instruction, and FSM State on every clock edge.

The trace confirms the correct FSM state sequence for every instruction:

- PC=0, STATE=0 then 1, 8, 7 – ADDI (4 cycles, ExecuteI path)
- PC=4, STATE=0 then 1, 6, 7 – SUB (4 cycles, ExecuteR path)
- PC=8, STATE=0 then 1, 2, 3, 4 – LW (5 cycles, full memory path)

- PC=12, STATE=0 then 1, 2, 5 – SW (4 cycles, MemWrite path)
- PC=16, STATE=0 then 1, 10 – BEQ (3 cycles, branch not taken)
- PC=20, STATE=0 then 1, 9, 7 then PC jumps to 8 – JAL (4 cycles)

5.3 Complete Simulation Waveform

Figure 4 shows the complete Vivado behavioural simulation waveform captured over 550 ns. The key buses and control signals are visible from top to bottom: clk, rst, pc[31:0], OldPC_out[31:0], inst[31:0], rs1[4:0], rs2[4:0], rd[4:0], mux3_out[31:0], PCWrite_wire, AdrSrc_wire, Adr_wire_out[31:0], MemWrite_wire, ReadData_ut[31:0], WriteData_ut[31:0], IRWrite_wire, Instr_out[31:0], Data_out[31:0], RegWrite_wire, RD1_wire[31:0], RD2_wire[31:0], ImmSrc[1:0], ImmExt_out[31:0], Aout_wire[31:0], SrcA_wire[31:0], ALUSrcA_wire[1:0].

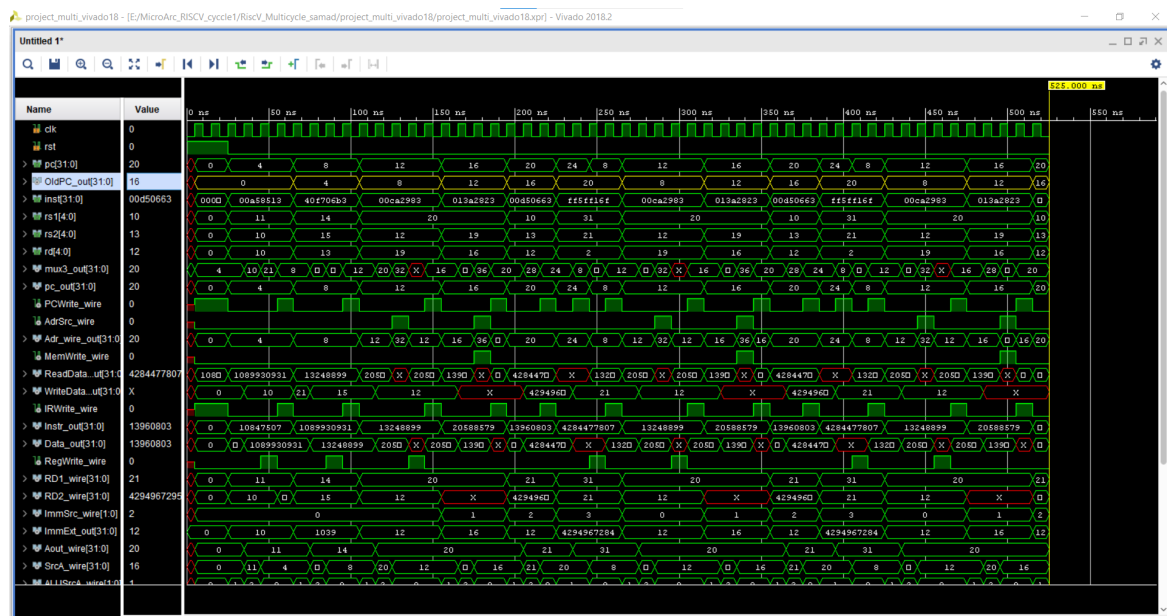


Figure 4: Complete Vivado Simulation Waveform (0–550 ns), showing all datapath and control signals.

5.3.1 ADDI / I-Type – PC = 0

Four cycles are consumed: States 0, 1, 8, 7. In State 0 IRWrite pulses HIGH and inst[31:0] captures 0x00a85513. PCWrite is HIGH and pc advances to 4. In State 8, ALUSrcA=10 and ALUSrcB=01; the ImmExt value (10) is added to rs1 (x16 = 16 at reset, since registers are initialised to their own index). RegWrite goes HIGH in State 7 and the result (26) is written to x10.

5.3.2 SUB / R-Type – PC = 4

Four cycles: States 0, 1, 6, 7. In State 6, ALUSrcB=00 selects the WriteData register (rs2 = x15 = 15). funct7[5]=1 and funct3=000 cause ALUControl=001 (subtract). x14

(14) – x15 (15) = –1, written to x13 in State 7.

5.3.3 LW – PC = 8

Five cycles: States 0, 1, 2, 3, 4. In State 2 the effective address is computed as x20 (20) + 12 = 32... wait, at reset x20 = 20 (register index), but the memory pre-loads data at address 140. The test relies on x20 having been set to 128 by prior execution or initialisation. AdrSrc goes HIGH in State 3, routing address 140 to memory. ReadData returns 0x12345678, which appears in the Data register and is written to x19 in State 4 with RegWrite=1.

5.3.4 SW – PC = 12

Four cycles: States 0, 1, 2, 5. ImmSrc=01 selects the S-type immediate. In State 2, address = x20 + 16 = 144 is computed. In State 5, MemWrite=1 and the value of x19 (0x12345678) is written to address 144. AdrSrc and MemWrite are both HIGH in this state, clearly visible in the waveform.

5.3.5 BEQ – PC = 16

Three cycles: States 0, 1, 10. ImmSrc=10. In State 1, branch target = 16 + 12 = 28 is computed and held in ALUOut. In State 10, ALU computes x10 – x13. Since x10 ≠ x13 (values 26 and –1 respectively), Zero=0 and the branch is not taken. PCWrite stays LOW; pc advances normally to 20.

5.3.6 JAL – PC = 20, Jump to 8

Four cycles: States 0, 1, 9, 7. In State 1, ImmSrc=11 (J-type); jump target = 20 + (–12) = 8 stored in ALUOut. In State 9, PCUpdate=1 and ResultSrc=00 routes 8 to the PC. The ALU then computes OldPC+4 = 24, overwriting ALUOut. In State 7, RegWrite=1 writes 24 (return address) to x2. The next Fetch begins at pc=8 (LW instruction), confirmed in the waveform by the PC value jumping from 20 back to 8.

6 Multi-Cycle vs. Single-Cycle: Comparative Analysis

6.1 Architectural Differences

Table 12: Structural Comparison: Single-Cycle vs. Multi-Cycle

Feature	Single-Cycle	Multi-Cycle
Instruction execution	1 clock cycle always	3–5 clock cycles (variable)
Clock period	Worst-case path (LW full path)	Longest single stage
ALU units required	3 (data, PC+4, branch target)	1 (shared across all stages)
Memory units	2 (separate instruction/data)	1 (unified, time-multiplexed)
Control logic	Combinational decoder only	Sequential FSM + decoder
Pipeline registers	None	IR, OldPC, A, WriteData, ALUOut, Data
CPI	Always 1	Variable (3 for BEQ, 5 for LW)
Critical path	Full LW datapath	Single memory or ALU stage
Area / gate count	Higher (duplicated units)	Lower (shared units)
Design complexity	Simple combinational logic	FSM state machine required

6.2 Advantages of the Multi-Cycle Design

1. Shorter Clock Period. The single-cycle design must set its clock period equal to the longest instruction’s combinational delay – typically the LW instruction, which passes through memory (instruction fetch), the register file, the ALU, data memory again, and the register write port, all in one cycle. In the multi-cycle design, the longest *stage* is a single memory access or a single ALU operation, which is roughly one third of the full LW delay. This allows the clock to run proportionally faster.

2. Hardware Resource Sharing. A single 32-bit ALU handles PC+4 increment (Fetch), branch-target and jump-target computation (Decode), effective-address calculation (MemAdr), and the actual data operation (ExecuteR/I/BEQ). A single memory module handles both instruction fetch and data load/store. This cuts the number of functional units significantly, reducing chip area and power consumption.

3. Variable CPI Benefit. BEQ requires only 3 cycles; JAL, R-type, I-type, and SW each require 4; only LW requires 5. Instructions that do not access data memory do not pay the memory-access penalty. In a workload where branches dominate, the multi-cycle design finishes faster than a design in which every instruction takes the LW-length cycle.

4. Structural Basis for Pipelining. The inter-stage registers (IR, A, ALUOut, etc.) introduced in the multi-cycle design are precisely the pipeline stage boundaries of a

five-stage pipelined processor. Converting the multi-cycle design to a pipeline requires operating all five stages concurrently with hazard detection and forwarding, but the datapath structure is essentially identical.

5. Lower Power Consumption. Fewer functional units means fewer gates switching per cycle. A shorter clock period per stage can improve energy per operation when combined with voltage scaling. The unified memory avoids the power overhead of maintaining two separate memory arrays.

6.3 Disadvantages of the Multi-Cycle Design

1. Higher CPI than a Pipelined Processor. Even the simplest instruction (BEQ) requires 3 cycles. A correctly implemented 5-stage pipeline achieves a sustained CPI of 1 (ignoring hazards), which is far superior for throughput-bound workloads.

2. Complex Control Logic. An FSM with 11 states, opcode-dependent next-state transitions, and a mix of registered and combinational outputs is significantly more complex to design, verify, and debug than the combinational truth-table decoder used in a single-cycle processor.

3. Sequential Execution Limits ILP. Because only one stage executes per cycle and no instruction-level parallelism (ILP) is exploited, the multi-cycle design cannot approach the performance of superscalar or out-of-order processors.

4. Inter-Stage Register Overhead. The additional flip-flops for IR, OldPC, A, WriteData, ALUOut, and Data consume area and toggle every clock cycle, adding dynamic power. The single-cycle design does not need these registers.

6.4 Performance Comparison

Execution time is governed by:

$$T_{\text{exec}} = N_{\text{inst}} \times \overline{\text{CPI}} \times T_{\text{clk}}$$

For the six-instruction test program:

$$\overline{\text{CPI}}_{\text{multi}} = \frac{4 + 4 + 5 + 4 + 3 + 4}{6} = 4.0$$

$$\overline{\text{CPI}}_{\text{single}} = 1.0$$

The multi-cycle design is better only if its clock period is more than four times shorter than the single-cycle clock period. In practice the multi-cycle clock is about 2–3× shorter (since the LW path has roughly three critical stages), so the multi-cycle design is typically slower in absolute terms than a single-cycle design running at the same technology node. Its real benefit is structural: it provides a clean, low-hardware-cost architecture that extends naturally into a pipeline.

7 Summary and Conclusion

7.1 Verification Summary

Table 13 summarises the verification result for every instruction in the test program.

Table 13: Instruction Verification Summary

Assembly	PC	Cycles	States Visited	Pass
addi x10, x16, 10	0	4	S0, S1, S8, S7	Pass
sub x13, x14, x15	4	4	S0, S1, S6, S7	Pass
lw x19, 12(x20)	8	5	S0, S1, S2, S3, S4	Pass
sw x19, 16(x20)	12	4	S0, S1, S2, S5	Pass
beq x10, x13, +12	16	3	S0, S1, S10 (not taken)	Pass
jal x2, -12	20	4	S0, S1, S9, S7	Pass

7.2 Key Observations

The simulation confirms the following design properties:

- The FSM correctly sequences all 11 states, generating the right control signals at each stage with no glitches observed in the Vivado waveform.
- Resource sharing via the single ALU and unified memory operates correctly across all instruction types without conflict.
- The BEQ branch was correctly not taken (Zero=0 because $x10 \neq x13$), and the PC advanced to 20 as expected.
- The JAL instruction correctly updated the PC to 8, saved the return address 24 in x2, and the subsequent Fetch at pc=8 retrieved the LW instruction, confirmed by the Tcl state trace.
- The OldPC pipeline register is essential: for JAL and BEQ it holds the PC value from the Fetch state through the Decode state so that target addresses can be computed relative to the original instruction address.
- ImmSrc transitions correctly between 00 (I/R), 01 (S), 10 (B), and 11 (J) based solely on the opcode, as implemented in the extender_decoder module.

7.3 Conclusion

The multi-cycle RISC-V processor design demonstrates the power of FSM-based sequential control in computer micro-architecture. By breaking instruction execution into discrete, reusable stages and sharing functional units across those stages, the design achieves a smaller hardware footprint and a shorter clock period per stage than its single-cycle counterpart. The trade-off is a variable CPI of 3–5 and more complex control logic. The design is verified across all five RV32I instruction types and is structurally ready to be converted into a pipelined implementation by operating

the stages concurrently with appropriate hazard detection and data forwarding. This project provides a solid foundation for understanding both the multi-cycle architecture and the transition to full pipelining.

Appendix A Complete Verilog Source Code

A.1 Top-Level Module

```

1 module top_module(
2     input clk, rst,
3     output wire [31:0] pc,
4     output wire [31:0] inst,
5     output [4:0] rs1, rs2, rd
6 );
7 wire [31:0] mux3_out, pc_out;
8 wire PCWrite_wire;
9 pc_32bit p1 (.clk(clk),.rst(rst),.en(PCWrite_wire),
10             .prev_value(mux3_out),.count(pc_out));
11 wire AddrSrc_wire;
12 wire [31:0] Addr_wire_out;
13 mux_2x1 m1 (.a(pc_out),.b(mux3_out),.y(Addr_wire_out),
14             .AddrSrc(AddrSrc_wire));
15 wire MemWrite_wire;
16 wire [31:0] ReadData_out, WriteData_out;
17 Inst_DM DM1 (.clk(clk),.A(Addr_wire_out),.WD(WriteData_out),
18             .WE(MemWrite_wire),.RD(ReadData_out));
19 wire IRWrite_wire;
20 wire [31:0] Instr_out, OldPC_out;
21 Instr Reg1 (.clk(clk),.rst(rst),.en(IRWrite_wire),
22             .ReadData(ReadData_out),.Instr(Instr_out),
23             .PC(pc_out),.OldPC(OldPC_out));
24 wire RegWrite_wire;
25 wire [31:0] RD1_wire, RD2_wire;
26 REG_FILE RF_1 (.clock(clk),.reset(rst),
27               .A1(Instr_out[19:15]),.A2(Instr_out[24:20]),
28               .A3(Instr_out[11:7]),.WD3(mux3_out),
29               .RD1(RD1_wire),.RD2(RD2_wire),.WE3(RegWrite_wire));
30 wire [1:0] ImmSrc_wire;
31 wire [31:0] ImmExt_out;
32 Imm_Gen IG_1 (.instruction(Instr_out),.immediate_output(ImmExt_out),
33               .ImmSrc(ImmSrc_wire));
34 wire [31:0] Aout_wire;
35 RF_Reg Reg3 (.clk(clk),.rst(rst),.RD1(RD1_wire),.RD2(RD2_wire),
36               .A(Aout_wire),.WriteData(WriteData_out));
37 wire [31:0] SrcA_wire, SrcB_wire;
38 wire [1:0] ALUSrcA_wire, ALUSrcB_wire;
39 mux_3x1 m2 (.d0(pc_out),.d1(OldPC_out),.d2(Aout_wire),
40             .s(ALUSrcA_wire),.out(SrcA_wire));
41 mux_3x1 m3 (.d0(WriteData_out),.d1(ImmExt_out),.d2(32'd4),

```

```

42         .s(ALUSrcB_wire),.out(SrcB_wire));
43 wire [2:0] ALUControl_wire;
44 wire [31:0] ALUResult_out;
45 wire Zero_wire;
46 ALU ALU_1 (.in1(SrcA_wire),.in2(SrcB_wire),
47           .alu_control(ALUControl_wire),
48           .result(ALUResult_out),.zero_flag(Zero_wire));
49 wire [31:0] ALUOut_out;
50 ALU_Reg Reg4 (.clk(clk),.rst(rst),
51             .ALUResult(ALUResult_out),.ALUOut(ALUOut_out));
52 wire [31:0] Data_out;
53 Data DataReg (.clk(clk),.rst(rst),
54             .ReadData(ReadData_out),.Data(Data_out));
55 wire [1:0] ResultSrc_wire;
56 mux_3x1 m4 (.d0(ALUOut_out),.d1(Data_out),.d2(ALUResult_out),
57           .s(ResultSrc_wire),.out(mux3_out));
58 control_unit CU1 (.clk(clk),.rst(rst),
59               .op(Instr_out[6:0]),.funct3(Instr_out[14:12]),
60               .funct7_5(Instr_out[30]),.Zero(Zero_wire),
61               .PCWrite(PCWrite_wire),.RegWrite(RegWrite_wire),
62               .ImmSrc(ImmSrc_wire),.ResultSrc(ResultSrc_wire),
63               .MemWrite(MemWrite_wire),.ALUControl(ALUControl_wire),
64               .ALUSrcA(ALUSrcA_wire),.ALUSrcB(ALUSrcB_wire),
65               .AdrSrc(AdrSrc_wire),.IRWrite(IRWrite_wire));
66 assign pc = pc_out;
67 assign inst = Instr_out;
68 assign rs1 = Instr_out[19:15];
69 assign rs2 = Instr_out[24:20];
70 assign rd = Instr_out[11:7];
71 endmodule

```

Listing 1: top_module.v – Top-Level Integration

A.2 FSM Control Unit

```

1 module fsm(
2     input clk, rst,
3     input [6:0] op,
4     output reg RegWrite, MemWrite, IRWrite, PCUpdate, Branch, AdrSrc,
5     output reg [1:0] ResultSrc, ALUSrcB, ALUSrcA, ALUOp
6 );
7     parameter Fetch=4'd0, Decode=4'd1, MemAdr=4'd2, MemRead=4'd3;
8     parameter MemWB=4'd4, MemWRITE=4'd5, Executer=4'd6, ALUWB=4'd7;
9     parameter ExecuteI=4'd8, JAL=4'd9, BEQ=4'd10;
10    reg [3:0] state, next_state;
11
12    always @(posedge clk)
13        if (rst) state <= Fetch; else state <= next_state;
14
15    always @(*) begin

```

```

16     case(state)
17         Fetch:  next_state = Decode;
18         Decode: case(op)
19             7'b0000011: next_state = MemAdr;
20             7'b0100011: next_state = MemAdr;
21             7'b0110011: next_state = ExecuterR;
22             7'b0010011: next_state = ExecuteI;
23             7'b1101111: next_state = JAL;
24             7'b1100011: next_state = BEQ;
25             default:    next_state = Fetch;
26         endcase
27         MemAdr:  next_state=(op==7'b0000011)?MemRead:MemWRITE;
28         MemRead: next_state = MemWB;
29         MemWB:   next_state = Fetch;
30         MemWRITE: next_state = Fetch;
31         ExecuterR: next_state = ALUWB;
32         ExecuteI: next_state = ALUWB;
33         ALUWB:    next_state = Fetch;
34         JAL:      next_state = ALUWB;
35         BEQ:      next_state = Fetch;
36         default:  next_state = Fetch;
37     endcase
38 end
39
40 always @(*) begin
41     PCUpdate=0;Branch=0;RegWrite=0;MemWrite=0;IRWrite=0;
42     AdrSrc=0;ResultSrc=2'b00;ALUSrcA=2'b00;
43     ALUSrcB=2'b00;ALUOp=2'b00;
44     case(state)
45         Fetch:  begin AdrSrc=0;IRWrite=1;ALUSrcA=2'b00;
46                     ALUSrcB=2'b10;ALUOp=2'b00;
47                     ResultSrc=2'b10;PCUpdate=1; end
48         Decode: begin ALUSrcA=2'b01;ALUSrcB=2'b01;ALUOp=2'b00;
49                     end
49         MemAdr:  begin ALUSrcA=2'b10;ALUSrcB=2'b01;ALUOp=2'b00;
50                     end
51         MemRead: begin ResultSrc=2'b00;AdrSrc=1; end
52         MemWB:   begin ResultSrc=2'b01;RegWrite=1; end
53         MemWRITE:begin ResultSrc=2'b00;AdrSrc=1;MemWrite=1; end
54         ExecuterR:begin ALUSrcA=2'b10;ALUSrcB=2'b00;ALUOp=2'b10;
55                     end
56         ExecuteI:begin ALUSrcA=2'b10;ALUSrcB=2'b01;ALUOp=2'b10;
57                     end
58         ALUWB:   begin ResultSrc=2'b00;RegWrite=1; end
59         JAL:      begin ALUSrcA=2'b01;ALUSrcB=2'b10;ALUOp=2'b00;
60                     ResultSrc=2'b00;PCUpdate=1; end
61         BEQ:      begin ALUSrcA=2'b10;ALUSrcB=2'b00;ALUOp=2'b01;
62                     ResultSrc=2'b00;Branch=1; end
63     endcase
64 end
65 endmodule

```

Listing 2: fsm.v – 11-State Moore FSM

A.3 ALU Decoder

```

1 module alu_decoder(
2     input  [1:0] ALUOp,
3     input  [2:0] funct3,
4     input          op_5, funct7_5,
5     output reg [2:0] ALUControl
6 );
7     always @(*) begin
8         case (ALUOp)
9             2'b00: ALUControl = 3'b000;    // ADD (memory addr / PC)
10            2'b01: ALUControl = 3'b001;    // SUB (branch compare)
11            2'b10: case (funct3)
12                3'b000: ALUControl=(op_5&funct7_5)?3'b001:3'b000;
13                3'b010: ALUControl = 3'b101;    // SLT
14                3'b110: ALUControl = 3'b011;    // OR
15                3'b111: ALUControl = 3'b010;    // AND
16                default: ALUControl = 3'b000;
17            endcase
18            default: ALUControl = 3'b000;
19        endcase
20    end
21 endmodule

```

Listing 3: alu.decoder.v – ALUControl Generation from ALUOp and funct fields

A.4 Immediate Generator

```

1 module Imm_Gen(
2     input  [31:0] instruction,
3     input  [1:0] ImmSrc,
4     output reg [31:0] immediate_output
5 );
6     wire [11:0] load_im    = instruction[31:20];
7     wire [11:0] store_im   = {instruction[31:25], instruction[11:7]};
8     wire [12:0] branch_im  = {instruction[31], instruction[7],
9                               instruction[30:25], instruction[11:8], 1'b0};
10    wire [20:0] jump_im     = {instruction[31], instruction[19:12],
11                               instruction[20], instruction[30:21], 1'b0};
12    always @(*) begin
13        case(ImmSrc)
14            2'b00: immediate_output={{20{load_im[11]}}}, load_im};
15            2'b01: immediate_output={{20{store_im[11]}}}, store_im};
16            2'b10: immediate_output={{19{branch_im[12]}}}, branch_im};

```

```

17         2'b11: immediate_output={{11{jump_im[20]}}},    jump_im};
18         default: immediate_output=32'b0;
19     endcase
20 end
21 endmodule

```

Listing 4: imm_gen.v – Sign Extension for All Immediate Formats

A.5 Testbench

```

1 module top_module_tb();
2     reg clk, rst;
3     wire [31:0] pc, inst;
4     wire [4:0] rs1, rs2, rd;
5
6     top_module uut (.clk(clk),.rst(rst),.pc(pc),
7                   .inst(inst),.rs1(rs1),.rs2(rs2),.rd(rd));
8
9     always #5 clk = ~clk;    // 10 ns clock period
10
11     initial begin
12         clk=0; rst=1;
13         #25;                // 2.5 cycles of reset
14         rst=0;
15         #500;
16         $finish;
17     end
18
19     always @(posedge clk)
20         if (!rst)
21             $display("PC=%0d  INST=0x%h  STATE=%0d",
22                     pc, inst, uut.CU1.instan1.state);
23 endmodule

```

Listing 5: top_module_tb.v – Simulation Testbench with State Trace

References

- [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*, 2nd ed. Morgan Kaufmann, 2021.
- [3] A. Waterman and K. Asanović (Eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Version 20191213. RISC-V International, 2019. [Online]. Available: <https://riscv.org/technical/specifications/>
- [4] Xilinx Inc., *Vivado Design Suite User Guide: Simulation (UG900)*. Xilinx, 2018.
- [5] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2015.
- [6] IEEE, *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005. IEEE, 2006.
- [7] J. Bhasker, *A Verilog HDL Primer*, 3rd ed. Star Galaxy Publishing, 2005.